

Átomos, Listas y Unificación

Robin Omar Buezo Díaz



Universidad San Carlos de Guatemala

Facultad de ingeniería.

Ingeniería en ciencias y sistemas

Indice

Contenido

Indice	1
Competencia(s)	2
Manejo de átomos y listas.....	2
Objetivo.....	2
Introducción.....	2
Contenido	2
Pongamos en Práctica la Teoría	3
Caso de Estudio 1.....	3
Caso de Estudio 2.....	3
Conclusiones	3
Principios de unificación de términos	3
Objetivo.....	3
Introducción.....	3
Contenido	3
Pongamos en Práctica la Teoría	4
Caso de Estudio 1.....	4
Caso de Estudio 2.....	4
Conclusión.....	4
Conclusión General	4
Referencias	5

Competencia(s)

Desarrolla hechos, reglas, expresiones y predicados recursivos mediante el uso de cláusulas, ciclos, listas, unificación y cortes en Tau Prolog para modelar bases de conocimiento y resolver problemas de inferencia lógica.

Manejo de átomos y listas

Objetivo

Identificar y manipular correctamente los átomos y listas en el contexto lógico de Prolog para representar datos y relaciones simples o agrupadas.

Introducción

Un átomo en Prolog es una constante simbólica indivisible. Las listas, en cambio, permiten agrupar múltiples elementos en una sola estructura. Ambos conceptos son esenciales para modelar información en sistemas lógicos.

Contenido

Los átomos son secuencias de caracteres que representan identificadores únicos. Se escriben en minúscula, como `perro`, `juan` o `activo`.

Las listas se representan entre corchetes:

- Lista vacía: `[]`
- Lista de un elemento: `[x]`
- Lista de varios: `[a, b, c]`

Prolog permite descomponer listas con la notación `[Cabeza | Cola]`, lo que facilita el trabajo recursivo sobre ellas.

Ejemplo básico:

```
prolog
CopiarEditar
es_lista([_ | _]).
es_vacia([]).
```

Pongamos en Práctica la Teoría

Caso de Estudio 1

En un sistema de seguimiento de cursos, se utiliza la lista [matematicas, fisica, logica] para representar materias inscritas.

Una regla puede verificar si una materia está en la lista usando patrón de descomposición.

Caso de Estudio 2

Para registrar ingredientes en recetas, se usa una lista como [huevo, harina, leche]. Con una regla recursiva se puede contar o verificar si un ingrediente está presente.

Conclusiones

El manejo de átomos y listas es esencial para representar información estructurada. Las listas, en particular, son útiles cuando se requiere manejar conjuntos dinámicos de datos.

Principios de unificación de términos

Objetivo

Aplicar correctamente el proceso de unificación para hacer coincidir estructuras lógicas durante la resolución de consultas o reglas.

Introducción

La unificación es el mecanismo central de Prolog. Permite comparar términos y asignar valores a variables si las estructuras coinciden. Esta operación se realiza de forma automática cuando se consulta o se ejecuta una regla.

Contenido

La unificación ocurre cuando dos términos pueden hacerse idénticos asignando valores a sus variables. Si esto es posible, Prolog continúa; si no, falla.

Ejemplo:

```
prolog
CopiarEditar
padre(juan, X) = padre(juan, pedro) → X = pedro
```

Pero si las estructuras no coinciden, la unificación falla:

```
prolog
CopiarEditar
padre(juan, X) ≠ madre(juan, X)
```

También se puede forzar la comparación explícita con el operador `=`, y evitarla con `\=`.

Pongamos en Práctica la Teoría

Caso de Estudio 1

En una base de datos familiar, se puede buscar si dos personas comparten padre usando:

```
padre(P, X), padre(P, Y), X \= Y
```

Esto unifica el padre pero obliga a que los hijos sean distintos.

Caso de Estudio 2

En un sistema de seguimiento de paquetes, se puede validar si el estado actual coincide con uno esperado mediante unificación directa:

```
estado(paquete123, entregado) = estado(paquete123, Estado)
```

Esto da como resultado `Estado = entregado`.

Conclusión

Comprender cómo funciona la unificación permite escribir reglas más precisas y aprovechar al máximo la capacidad lógica de Prolog. Es una técnica clave para la resolución automática de problemas en sistemas declarativos.

Conclusión General

El trabajo con átomos, listas y unificación permite modelar estructuras de datos simples y compuestas, así como evaluar condiciones lógicas. Estas herramientas básicas son el punto de partida para construir aplicaciones declarativas sólidas y eficientes.

Referencias

Bratko, I. (2012). *Prolog Programming for Artificial Intelligence*. Pearson
Clocksin, W. F., & Mellish, C. S. (2003). *Programming in Prolog*. Springer
Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*. Pearson